1.3.4. System monitoring

Once your code goes into production, you should have some type of system that looks at it to make sure it's working and that tells you if something goes wrong, such as whether a website becomes unresponsive or if your mobile app starts to crash every time a user opens it. If you're doing a really great job with your testing, you likely are paying attention to your system monitoring during your testing phase as well making sure you don't see any issues from a support perspective before your code goes into production.

The problem with system monitoring is that it's largely reactive, not proactive. Ideally all your data should be as close to the development cycle as possible, in which case you'll be able to react to it as a team quickly. By the time you're looking at system-monitoring data in a production environment your sprint is done, the code is out, and if there's a problem, then you're usually scrambling to fix it rather than learning and reacting to it with planned work.

Let's look at ways to mitigate this problem. One way is to use system monitoring as you test your code before it gets to production. There are a number of ways your code can make it to production. Typically you see something like the flow shown in figure 1.9, where a team works in STET local development environment and pushes changes to an integration environment, and where multiple change sets are tested together and a QA environment verifies what the end user or customer will get before you ship your code to production.

Figure 1.9. Typical environment flow in the path to production